So You Think You Know Python?

Easter eggs plus other interesting bits.

Jordan Duabe
2025-03-05

whoami

Jordan Duabe

Software Engineer, Hobbyist Pythonista

me@jduabe.dev

@j4ckofalltrades@infosec.exchange

which python

The CPython repo is hosted on GitHub.

Python 3.9-3.14: uv python list, uv run –python 3.14 python

Python 2.7, 3.0-3.8: Docker Official Image, docker run -it –rm python:3.8 python

Python 2.0-2.6: Python Source releases, may need to compile from source

Python 1.0.1-1.6, 2.0: Legacy Python releases, may need to compile from source

Vintage Python 0.9.0, 1.0.1, 1.1, 1.2, 1.3: Dockerfiles and source code

Hello world!

< Python 3.11

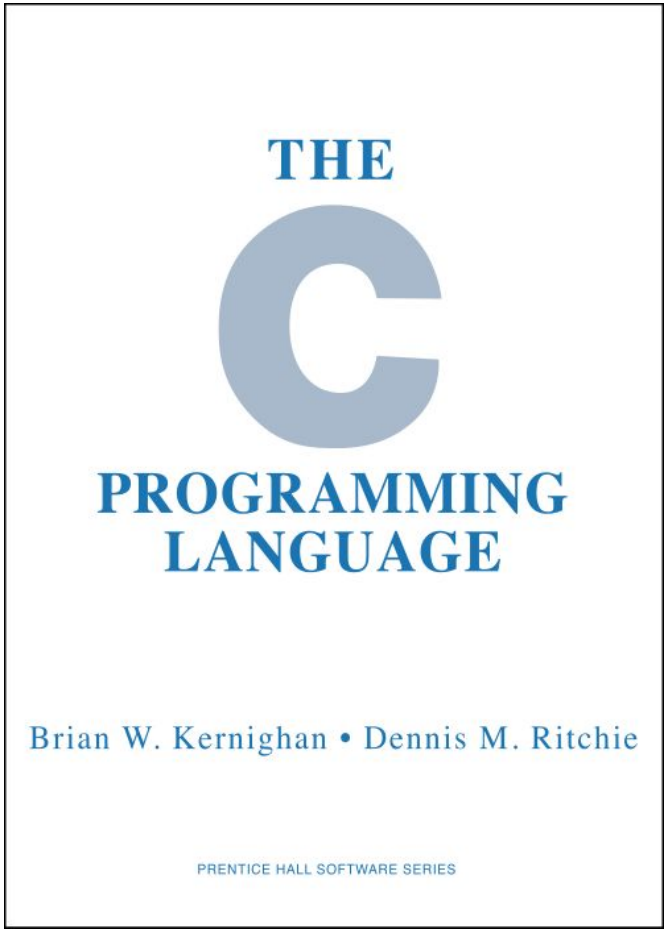
```
>>> import __hello__
Hello world!
>>> import __phello__
Hello world!
```

>= Python 3.11

```
>>> import __hello__
>>> __hello__.main()
Hello world!
>>> import __phello__
>>> __phello__.main()
Hello world!
```

The OG

```c
main()
{
    printf("hello, world\n");
}
```



Spam, ham and eggs

>= Python 3.11

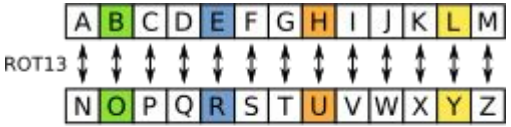
```
>>> import __phello__.spam
>>> __phello__.spam.main()
Hello world!
>>> import __phello__.ham
>>> import __phello__.ham.eggs
```

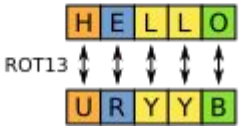
Zen of Python

```
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

Zen of Python (source)

ROT13 is a simple letter substitution cipher that replaces a letter with the 13th letter after it in the Latin alphabet (A becomes N, B becomes O).





Lib/this.py

```python
s = """Gur Mra bs Clguba, ol Gvz Crgref

Ornhgvshy vf orggre guna htyl.
Rkcyvpvg vf orggre guna vzcyvpvg.
Fvzcyr vf orggre guna pbzcyrk.
Pbzcyrk vf orggre guna pbzcyvpngrq.
Syng vf orggre guna arfgrq.
Fcnefr vf orggre guna qrafr.
Ernqnovyvgl pbhagf.
Fcrpvny pnfrf nera'g fcrpvny rabhtu gb oernx gur ehyrf.
Nygubhtu cenpgvpnyvgl orngf chevgl.
Reebef fubhyq arire cnff fvyragyl.
Hayrff rkcyvpvgyl fvyraprq.
Va gur snpr bs nzovthvgl, ershfr gur grzcgngvba gb thrff.
Gurer fubhyq or bar-- naq cersrenoyl bayl bar --boivbhf jnl gb qb vg.
Nygubhtu gung jnl znl abg or boivbhf ng svefg hayrff lbh'er Qhgpu.
Abj vf orggre guna arire.
Nygubhtu arire vf bsgra orggre guna *evtug* abj.
Vs gur vzcyrzragngvba vf uneq gb rkcynva, vg'f n onq vqrn.
Vs gur vzcyrzragngvba vf rnfl gb rkcynva, vg znl or n tbbq vqrn.
Anzrfcnprf ner bar ubaxvat terng vqrn -- yrg'f qb zber bs gubfr!"""

d = {}
for c in (65, 97):
    for i in range(26):
        d[chr(i+c)] = chr((i+13) % 26 + c)

print("".join([d.get(c, c) for c in s]))
```

The Python Way

"The Python Way"

Tim Peters tim_one at email.msn.com
Fri Jun 4 01:15:13 EDT 1999

- Previous message (by thread): "The Python Way"
- Next message (by thread): "The Python Way"
- **Messages sorted by:** [date] [thread] [subject] [author]

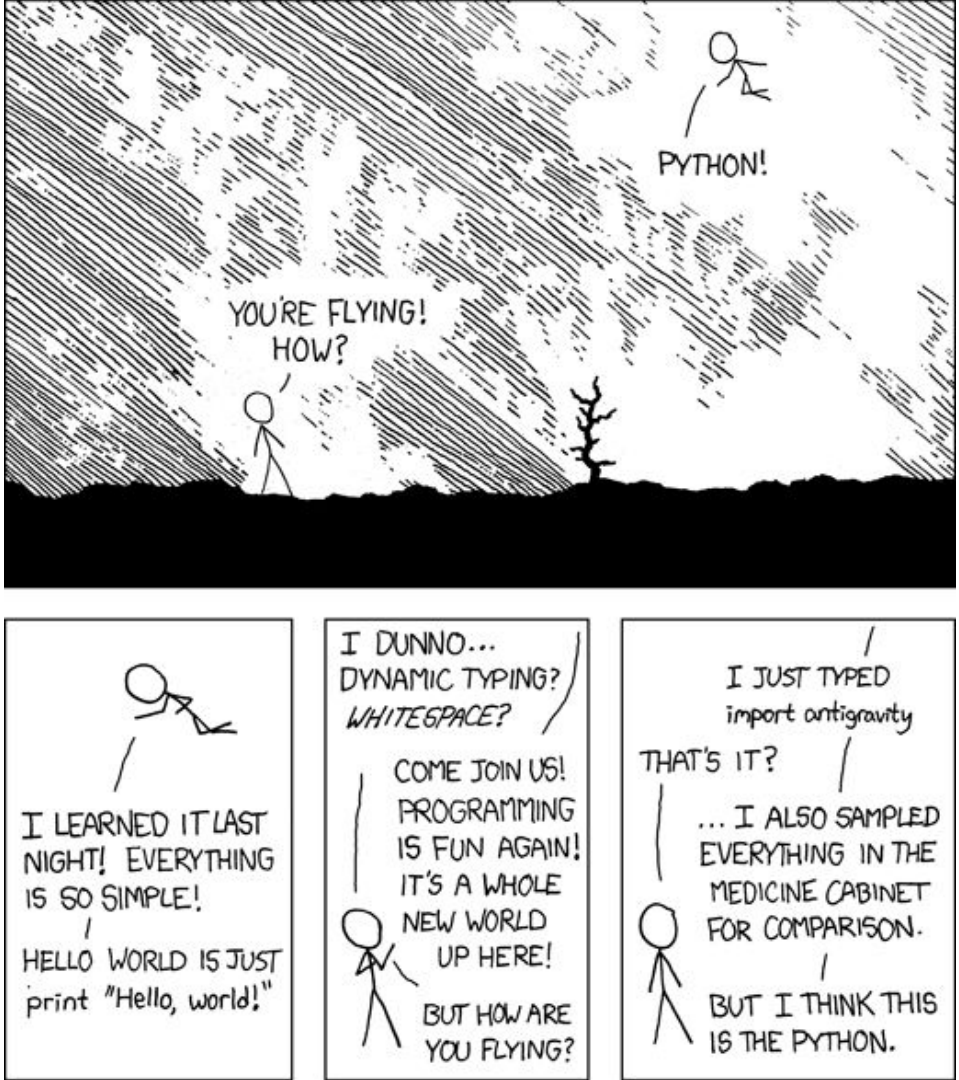
> but the more I use and learn about the language, the more I find myself
> appreciating the nice balance and heft Guido gave to it. Yet there doesn't
> seem to be a single document that sums up that "aesthetic," but rather it
> tends to appear piecemeal, over time, mostly in the Wisdom of Chairman
> Tim.

Eh? I make shit up as I go along -- & Guido often appears to think that my fabrications would be good positions for him to have pretended he had all along too.

OTOH, after all these years, I *still* have to lecture him about his true vision of what Python should be ... although he learns pretty fast, for a Dutch guy <wink>.

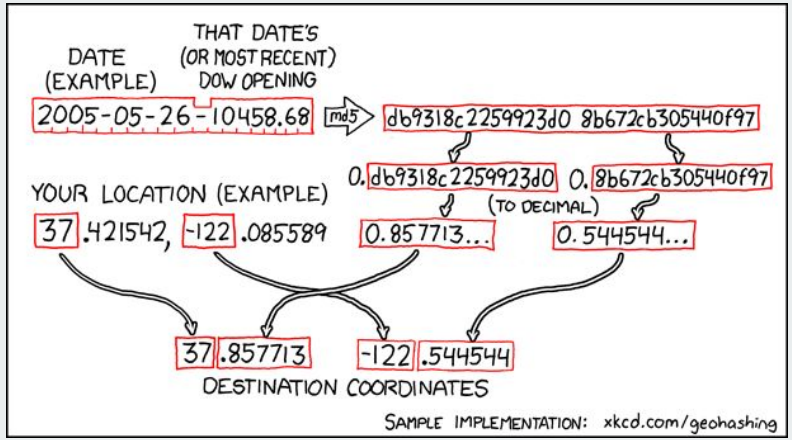
XKCD 353

```
>>> import antigravity
```



XKCD 426

```
>>> from antigravity import this
```



Lib/antigravity.py

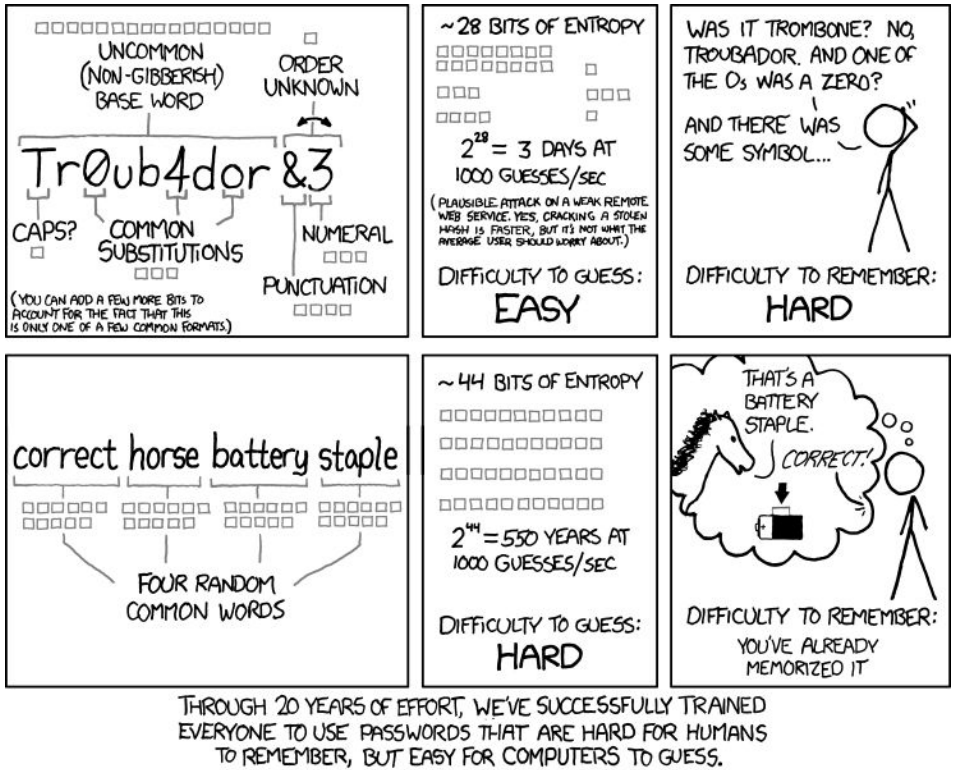
```python
import webbrowser
import hashlib

webbrowser.open("https://xkcd.com/353/")

def geohash(latitude, longitude, datedow):
    '''Compute geohash() using the Munroe algorithm.

    >>> geohash(37.421542, -122.085589, b'2005-05-26-10458.68')
    37.857713 -122.544543

    '''
    # https://xkcd.com/426/
    h = hashlib.md5(datedow, usedforsecurity=False).hexdigest()
    p, q = [('%f' % float.fromhex('0.' + x)) for x in (h[:16], h[16:32])]
    print('%d%s %d%s' % (latitude, p[1:], longitude, q[1:]))
```

XKCD 936

Generate an [XKCD-style passphrase]():

.. testcode::

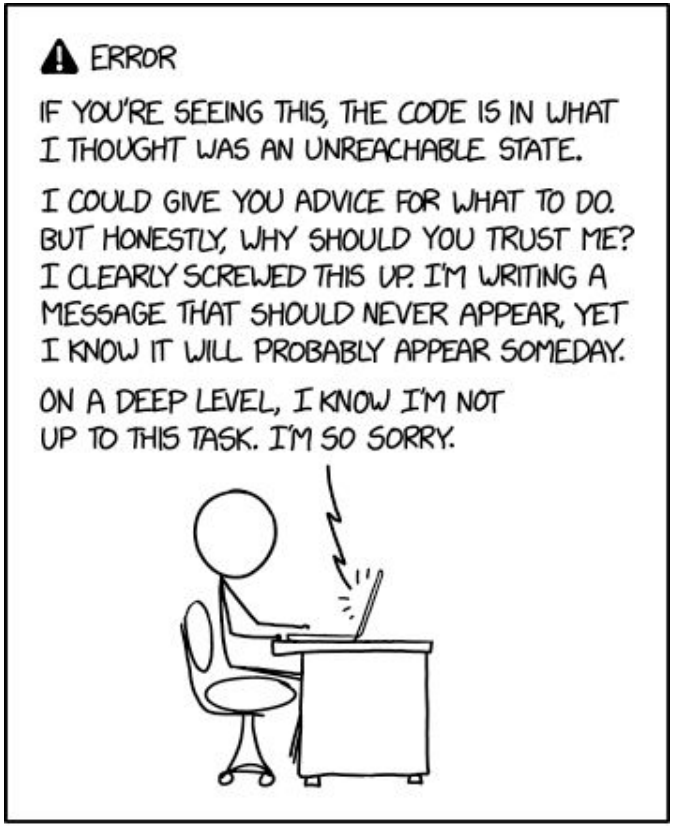
```
import secrets
# On standard Linux systems, use a convenient dictionary file.
# Other platforms may need to provide their own word-list.
with open('/usr/share/dict/words') as f:
    words = [word.strip() for word in f]
    password = ' '.join(secrets.choice(words) for i in range(4))
```



XKCD 2200

Include/pymacro.h

```
#if defined(RANDALL_WAS_HERE)
#  define Py_UNREACHABLE() \
    Py_FatalError( \
        "If you're seeing this, the code is in what I thought was\n" \
        "an unreachable state.\n\n" \
        "I could give you advice for what to do, but honestly, why\n" \
        "should you trust me?  I clearly screwed this up.  I'm writing\n" \
        "a message that should never appear, yet I know it will\n" \
        "probably appear someday.\n\n" \
        "On a deep level, I know I'm not up to this task.\n" \
        "I'm so sorry.\n" \
        "https://xkcd.com/2200")
#elif defined(Py_DEBUG)
#  define Py_UNREACHABLE() \
    Py_FatalError( \
        "We've reached an unreachable state. Anything is possible.\n" \
        "The limits were in our heads all along. Follow your dreams.\n" \
        "https://xkcd.com/2200")
```



NEVER WRITE ERROR MESSAGES TIRED.

Brace yourselves

```
>>> from __future__ import braces
  File "<python-input-0>", line 1
    from __future__ import braces
                            ^^^^^^
SyntaxError: not a chance
```

```
// Allman
while (x == y)
{
    foo();
    bar();
}

// K&R
while (x == y) {
    foo();
    bar();
}

// GNU
while (x == y)
  {
    foo();
    bar();
  }
```

April Fools

```
>>> from __future__ import barry_as_FLUFL
>>> 1 <> 2
True
>>> 1 ≠ 2
  File "<python-input-11>", line 1
    1 ≠ 2
      ^^
SyntaxError: with Barry as BDFL, use '<>' instead of '≠'
```

Not Equals

```
# Python 3
>>> 1 ≠ 2
True
>>> 1 <> 2
  File "<python-input-16>", line 1
    1 <> 2
      ^^

# Python 2
>>> 1 ≠ 2
True
>>> 1 <> 2
True
```

Hash Info

```
>>> import sys
>>> sys.hash_info
sys.hash_info(width=64, modulus=2305843009213693951, inf=314159, nan=0,
imag=1000003, algorithm='siphash13', hash_bits=64, seed_bits=128, cutoff=0)
```

To Infinity and NaN

```
# Python 3
>>> hash(float("Infinity")) # pi
314159
>>> hash(float("-Infinity"))
-314159
>>> hash(float("NaN"))
8779977821219

# Python 2
>>> hash(float("Infinity"))
314159
>>> hash(float("-Infinity")) # euler's number
-271828
>>> hash(float("Nan"))
0
```

Hashing numeric types

```
>>> import decimal
>>> hash(1_000_000) == hash(1_000_000.0) == hash(decimal.Decimal(1_000_000.0))
True
>>> hash(1_000_000_000_000_000_000)
1000000000000000000
>>> hash(1_000_000_000_000_000_000_000)
1569977010470519217
>>> hash(1_000_000_000_000_000_000_000_000)
2003764205207330320
```

Hashing strings

```
>>> import sys
>>> sys.hash_info.algorithm # ≥ Python 3.13
'siphash13'
>>> hash("PythonWA")
2905548310570534902

>>> hash("PythonWA") # ≥ Python 3.13
-8824279633306705447 # different per run

>>> import sys
>>> sys.hash_info.algorithm # ≥ Python 3.4
'siphash24'
>>> hash("PythonWA")
3262745232486003013

>>> hash("PythonWA") # ≥ Python 3.4
8422715740029748764  # different per run

>>> hash("PythonWA") # < Python 3.4
-5899562345255888120 # FNV (Fowler-Noll-Vo)

>>> hash("PythonWA") # same hash across runs
-5899562345255888120
```

Not for the faint of heart

```
>>> import types
>>> help(types.CodeType)

Help on class code in module builtins:

class code(object)
 |  code(
 |      argcount,
 |      posonlyargcount,
 |      kwonlyargcount,
 |      nlocals,
 |      stacksize,
 |      flags,
 |      codestring,
 |      constants,
 |      names,
 |      varnames,
 |      filename,
 |      name,
 |      qualname,
 |      firstlineno,
 |      linetable,
 |      exceptiontable,
 |      freevars=(),
 |      cellvars=(),
 |      /
 |  )
 |
 |  Create a code object.  Not for the faint of heart.
```

PEG Parser

Python 3.9

```
>>> __peg_parser__
  File "<stdin>", line 1
    __peg_parser__
    ^
SyntaxError: You found it!
```

NOT like this

```
>>> True | False
True
>>> 1 | 0
1
>>> True & False
False
>>> 1 & 0
0
>>> ~True
<python-input-13>:1: DeprecationWarning: Bitwise inversion '~' on
bool is deprecated and will be removed in Python 3.16. This
returns the bitwise inversion of the underlying int object and is
usually not what you expect from negating a bool. Use the 'not'
operator for boolean negation or ~int(x) if you really want the
bitwise inversion of the underlying int.
-2
>>> ~False
<python-input-14>:1: DeprecationWarning: Bitwise inversion '~' on
bool is deprecated and will be removed in Python 3.16. This
returns the bitwise inversion of the underlying int object and is
usually not what you expect from negating a bool. Use the 'not'
operator for boolean negation or ~int(x) if you really want the
bitwise inversion of the underlying int.
-1
```

DeprecationWarnings are disabled by default

```python
resp = urllib3.request("GET", "https://example.com")

# Deprecated APIs
resp.getheader("Content-Length")
resp.getheaders()

# Recommended APIs
resp.headers.get("Content-Length")
resp.headers
```

src/urllib3/response.py

```python
    # Compatibility methods for http.client.HTTPResponse
    def getheaders(self) -> HTTPHeaderDict:
        warnings.warn(
            "HTTPResponse.getheaders() is deprecated and will be removed "
            "in urllib3 v2.1.0. Instead access HTTResponse.headers directly.",
            category=DeprecationWarning,
            stacklevel=2,
        )
        return self.headers

    def getheader(self, name: str, default: Optional[str] = None) -> Optional[str]:
        warnings.warn(
            "HTTPResponse.getheader() is deprecated and will be removed "
            "in urllib3 v2.1.0. Instead use HTTResponse.headers.get(name, default).",
            category=DeprecationWarning,
            stacklevel=2,
        )
        return self.headers.get(name, default)
```

Is midnight falsy?

<= Python 3.4

```
>>> import datetime
>>> bool(datetime.time(0, 0, 0)) # midnight UTC
False
>>> bool(datetime.time(0, 0, 1))
True
```

>= Python 3.5

```
>>> import datetime
>>> bool(datetime.time(0, 0, 0)) # midnight UTC
True
>>> bool(datetime.time(0, 0, 1))
True
```

Don’t leave me hanging

r-strings can’t end with an odd number of backslashes: the unpaired backslash at the end escapes the closing quote character, leaving an unterminated string.

```
>>> r"Why can't raw strings end with a backslash?"
"Why can't raw strings end with a backslash?"
>>> r"Why can't raw strings end with a backslash? \"
  File "<python-input-10>", line 1
    r"Why can't raw strings end with a backslash? \"
    ^
```

Thank You